

Day 1: Introduction to Python and Cybersecurity

1. What is Python?

Python is a high-level, general-purpose programming language created by Guido van Rossum and first released in 1991. It is designed to be readable, so the code looks close to plain English.

Why beginners and professionals both use it:

- **Simple syntax:** fewer lines of code than C, C++, or Java for the same task.
- **Interpreted:** you run code line by line, which makes testing and debugging fast.
- **Huge library ecosystem:** ready-made modules for networking, files, encryption, web requests, and data analysis.
- **Cross-platform:** the same script runs on Windows, Linux, and macOS.
- **Large community:** lots of tutorials, tools, and help.

A first program:

```
python
```

```
print("Hello, Cybersecurity!")
```

Core concepts you will meet in the coming days: variables, data types (strings, integers, lists, dictionaries), conditions (if/else), loops (for/while), functions, modules, and file handling.

2. Uses of Python in Cybersecurity

Python is probably the most popular language in security because you can build a working tool in minutes.

Common uses:

Area	What Python does	Example
Network scanning	Checks which hosts and ports are open on systems you own	Writing a simple port checker with the socket library
Log analysis	Parses large log files to find suspicious activity	Detecting repeated failed logins
Automation	Replaces repetitive manual tasks	Auto-generating security reports
Password security	Checks password strength and hashing	Testing hash algorithms with hashlib
Packet analysis	Reads and inspects network traffic	scapy for analyzing captured packets
Web security testing	Sends HTTP requests to test your own apps	requests library, header checks
Digital forensics	Extracts data from files and disk images	Reading file metadata
Malware analysis	Analyzes suspicious files safely in isolated labs	Extracting strings, computing file hashes
Threat intelligence	Pulls data from security APIs	Checking if an IP is reported as malicious
Cryptography	Encrypts and decrypts data	cryptography library

Both defenders (blue team) and authorized testers (red team) use Python. This course focuses on building defensive skills and understanding how attacks work so you can protect systems.

3. Ethical Hacking and Legal Permission

Ethical hacking means testing the security of a system with the owner's explicit permission, in order to find weaknesses and fix them before criminals exploit them. Ethical hackers are also called penetration testers or white-hat hackers.

Types of hackers:

- **White hat:** authorized, works to improve security.
- **Black hat:** unauthorized, malicious or for personal gain (illegal).
- **Grey hat:** no permission, but no malicious intent. Still illegal.

The golden rule: permission first. Accessing or testing a computer system without authorization is a crime, even if you mean well and even if you cause no damage. In India, for example, the **Information Technology Act, 2000** (notably Sections 43 and 66) covers unauthorized access, and many countries have comparable laws (the CFAA in the US, the Computer Misuse Act in the UK).

What legal permission looks like in professional work:

- A signed **contract or authorization letter**
- A defined **scope** (which systems, IP ranges, and apps may be tested)
- **Rules of engagement** (allowed techniques, testing times, who to contact)
- A **non-disclosure agreement (NDA)**
- A clear process for **reporting findings**

Safe places to practice:

- Your own computer and your own virtual machines
- Intentionally vulnerable practice apps (e.g., DVWA, OWASP Juice Shop)
- Legal platforms such as TryHackMe, Hack The Box, and Capture The Flag (CTF) competitions
- Bug bounty programs, strictly within each program's stated rules

Never practice on: your college or office network, a friend's device, public websites, or any system you don't own, unless you have written permission.

4. Course Tools and Rules

Your instructor may adjust this list, but a typical setup looks like this.

Tools

1. **Python 3 (3.10 or newer):** the language itself. Check your install with `python --version`.
2. **Code editor/IDE:** VS Code or PyCharm Community Edition for writing and running code.
3. **pip:** Python's package manager, used to install libraries such as requests and scrapy.

4. **Virtual environments (venv):** keep each project's libraries separate so nothing conflicts.
5. **Virtualization software:** VirtualBox or VMware to run an isolated lab machine (for example Kali Linux or Ubuntu). A mistake inside a VM won't harm your main system.
6. **Wireshark:** a network traffic analyzer, used on your own lab network.
7. **Nmap:** a network scanner, used only on systems you own or are authorized to scan.
8. **Git and GitHub:** to save and version your code.
9. **Notes tool:** a notebook or app to document what you do and learn, a habit professionals follow.

Rules

1. **Authorized targets only.** Use only the lab machines and practice platforms provided in class.
2. **No attacks outside the lab.** Never test tools on college, home, or public networks you don't control.
3. **No malicious software.** You will not create or distribute viruses, ransomware, or tools meant to harm real people.
4. **Work in isolation.** Run experiments inside your virtual machine, with network settings your instructor approves.
5. **Respect privacy.** Don't access, collect, or share anyone's personal data.
6. **Report, don't exploit.** If you accidentally find a real vulnerability somewhere, report it responsibly instead of using it.
7. **Academic honesty.** Write your own code, understand it, and credit your sources.
8. **Document your work.** Keep notes of commands run, results, and lessons learned.
9. **Ask before you try.** If you're unsure whether something is allowed, ask your instructor first.
10. **Follow the law and the code of ethics.** Violating these rules can lead to removal from the course and legal consequences.

Practice task: install Python and VS Code, run `print("Hello, Cybersecurity!")`, and write 3 to 4 sentences on why permission matters in ethical hacking.

If you'd like, I can turn this into a Word document or slide deck, or prepare Day 2 (Python basics: variables, data types, and input/output).

Day 2: Python Installation and First Program

1. Installing Python and VS Code

Step 1: Install Python

1. Go to the official site: python.org/downloads
2. Download the latest Python 3 version (3.10 or newer).
3. Run the installer.
 - **Windows:** tick **"Add Python to PATH"** on the first screen before clicking *Install Now*. Skipping this is the most common beginner mistake.
 - **macOS:** open the downloaded .pkg file and follow the prompts.
 - **Linux:** Python is usually preinstalled. If not, run `sudo apt install python3 python3-pip`.
4. Verify the installation by opening a terminal (Command Prompt, PowerShell, or Terminal) and typing:

bash

`python --version`

On macOS/Linux you may need:

bash

`python3 --version`

You should see something like Python 3.12.x.

Step 2: Install VS Code

1. Go to code.visualstudio.com and download the version for your OS.
2. Install it with the default options.
3. Open VS Code and click the **Extensions** icon (four squares on the left sidebar).
4. Search for **Python** (published by Microsoft) and click **Install**.

Step 3: Quick check that everything works

Open VS Code, press `Ctrl + Shift + P` (`Cmd + Shift + P` on Mac), type **Python: Select Interpreter**, and choose the Python version you just installed.

2. Creating a .py File

A Python program is a plain text file that ends with .py.

In VS Code:

1. Create a folder for the course, for example `python-cybersecurity`.
2. In VS Code, go to **File** → **Open Folder** and select it.

3. Click the **New File** icon in the Explorer panel.
4. Name it hello.py. The .py extension is what tells the computer it is Python.
5. Type your code and save with Ctrl + S.

Running the file:

- Click the ► **Run** button at the top right of VS Code, **or**
- Open the terminal in VS Code (Ctrl + `) and type:

```
bash
```

```
python hello.py
```

Naming tips: use lowercase letters and underscores (port_checker.py), avoid spaces, and never name your file after a Python module (for example, don't call a file random.py).

3. Using print()

print() is a built-in function that displays output on the screen. It is how you see what your program is doing.

Basic use:

```
python
```

```
print("Hello, Cybersecurity!")
```

```
print("Welcome to Day 2")
```

Output:

```
Hello, Cybersecurity!
```

```
Welcome to Day 2
```

Key points:

- Text must be inside quotes, either "double" or 'single'.
- Each print() starts a new line by default.
- Numbers don't need quotes: print(2026).

Printing multiple items (separated by a space automatically):

```
python
```

```
print("Port", 80, "is open")
```

Output: Port 80 is open

Special characters:

```
python
```

```
print("Line one\nLine two") # \n = new line
```

```
print("Name:\tAlex") # \t = tab
```

```
print("She said \"hi\"")    # \" = quote inside quotes
```

Changing the ending and separator:

```
python
```

```
print("Scanning", end="...")
```

```
print("done")
```

```
# Output: Scanning...done
```

```
print("192", "168", "1", "1", sep=".")
```

```
# Output: 192.168.1.1
```

Multi-line text:

```
python
```

```
print("""Security Report
```

```
-----
```

```
Status: Safe""")
```

Common errors:

Mistake	Error you'll see	Fix
<code>print("Hello)</code>	<code>SyntaxError: unterminated string literal</code>	Close the quote
<code>Print("Hello")</code>	<code>NameError: name 'Print' is not defined</code>	Python is case-sensitive: use <code>print</code>
<code>print "Hello"</code>	<code>SyntaxError</code>	Python 3 needs parentheses

Reading error messages carefully is a core skill. They tell you the file, the line number, and the type of problem.

4. Writing Comments

Comments are notes for humans. Python ignores them completely. Good comments explain **why** the code exists, which is vital in security work where scripts get reviewed and shared.

Single-line comments start with `#`:

```
python
```

```
# This program prints a welcome message
```

```
print("Hello, Cybersecurity!") # inline comment after code
```

Multi-line comments (use several `#` lines, or a triple-quoted block):

```
python
```

```
# Author: Your Name
```

```
# Date: Day 2
```

```
# Purpose: First Python program
```

```
"""
```

```
This is a longer note
```

```
that spans multiple lines.
```

```
"""
```

Uses of comments:

- Explain what a section of code does
- Add your name, date, and purpose at the top of a file
- Temporarily disable code while testing:

```
python
```

```
# print("This line will not run")
```

Good vs. bad comments:

```
python
```

```
# Bad: says the obvious
```

```
x = 5 # sets x to 5
```

```
# Good: explains the reason
```

```
timeout = 5 # seconds to wait before giving up on a connection
```

Putting It All Together: Your First Program

Create day2_program.py:

```
python
```

```
# Program: Day 2 first program
```

```
# Author: Your Name
```

```
# Purpose: Practice print() and comments
```

```
print("=" * 30)
```

```
print(" Python for Cybersecurity")
```

```
print("=" * 30)
```

```
print("Student:", "Your Name")
```

```
print("Day:", 2)

print("Goal: Learn Python safely and ethically")
```

```
# print("This line is disabled by a comment")
```

```
print("Setup complete!")
```

Output:

```
=====

Python for Cybersecurity

=====
```

Student: Your Name

Day: 2

Goal: Learn Python safely and ethically

Setup complete!

("=" * 30 repeats the character 30 times, a handy trick for drawing lines.)

Troubleshooting

- **'python' is not recognized:** Python wasn't added to PATH. Reinstall and tick the PATH option, or use `py --version` on Windows.
- **VS Code can't find Python:** re-select the interpreter with *Python: Select Interpreter*.
- **Nothing happens when you run:** make sure you saved the file (Ctrl + S) and you're in the correct folder.
- **Indentation error:** don't put spaces before print at the start of a line.

Day 3: Variables and Data Types

1. Creating Variables

A **variable** is a named container that stores a value in memory. Think of it as a labeled box: you put something in, and later you can use the label to get it back.

Creating a variable uses the assignment operator =:

```
python
```

```
target_ip = "192.168.1.10"
```

```
port = 80
```

Python has no separate "declare" step. The variable is created the moment you assign a value. Note that = means "store the value on the right into the name on the left," not "equals" as in math.

Using variables:

```
python
```

```
username = "admin"
```

```
print(username)          # admin
```

```
print("User:", username)  # User: admin
```

Changing a value (variables can be reassigned at any time):

```
python
```

```
attempts = 1
```

```
attempts = 2
```

```
attempts = attempts + 1    # now 3
```

```
print(attempts)
```

Assigning multiple variables at once:

```
python
```

```
host, port = "localhost", 8080    # two values, two names
```

```
a = b = c = 0                    # same value to all three
```

Why variables matter in security scripts: instead of typing an IP address or port 20 times, you store it once and reuse it. If it changes, you edit one line.

2. Data Types: int, float, str, and bool

Every value in Python has a **type**, which decides what you can do with it.

int (integer): whole numbers

```
python
```

```
port = 443
```

```
failed_logins = 0
```

```
temperature = -5
```

Used for: port numbers, counters, number of attempts.

float: numbers with a decimal point

```
python
```

```
cpu_usage = 72.5
```

```
timeout = 2.5
```

```
pi = 3.14159
```

Used for: percentages, time measurements, scores.

str (string): text

```
python
```

```
name = "Alex"
```

```
protocol = 'HTTPS'
```

```
message = "Port 22 is open"
```

Strings go inside single or double quotes. Used for: usernames, IP addresses, messages, file paths.

Important: "80" (in quotes) is a **string**, while 80 is an **integer**. They behave differently:

```
python
```

```
print(80 + 20)    # 100 (math)
```

```
print("80" + "20") # 8020 (text joined together)
```

bool (boolean): True or False

```
python
```

```
is_open = True
```

```
is_admin = False
```

Only two values exist, and both must start with a capital letter (True, not true). Used for: yes/no decisions, such as "is the port open?" or "is the password strong?"

Quick reference

Type	Example	Meaning	Security use
int	22	Whole number	Port number
float	0.75	Decimal number	Risk score
str	"admin"	Text	Username

bool	True	True/False	Is service running?
------	------	------------	---------------------

Working with each type

Numbers can do math:

python

```
print(10 + 3) # 13 addition
```

```
print(10 - 3) # 7 subtraction
```

```
print(10 * 3) # 30 multiplication
```

```
print(10 / 3) # 3.3333333333333335 division (always float)
```

```
print(10 // 3) # 3 floor division
```

```
print(10 % 3) # 1 remainder
```

```
print(2 ** 3) # 8 power
```

Strings can be joined and repeated:

python

```
first = "Cyber"
```

```
second = "Security"
```

```
print(first + second) # CyberSecurity
```

```
print(first + " " + second) # Cyber Security
```

```
print("-" * 10) # -----
```

Mixing a string with a number causes an error:

python

```
print("Port: " + 80) # TypeError
```

```
print("Port: " + str(80)) # works: Port: 80
```

```
print("Port:", 80) # also works
```

Converting between types (type casting):

python

```
int("25") # 25
```

```
float("3.5") # 3.5
```

```
str(100) # "100"
```

```
bool(0) # False (0, "", and empty values are False)
```

```
bool(1) # True (any non-zero number or non-empty text is True)
```

3. Variable Naming Rules

Rules (break these and you get an error)

1. Must start with a **letter or underscore** (`_`), never a digit.
2. Can contain only **letters, digits, and underscores**. No spaces, hyphens, or symbols like `@`, `$`, `!`.
3. **Case-sensitive**: `port`, `Port`, and `PORT` are three different variables.
4. Cannot be a **reserved keyword** such as `if`, `for`, `while`, `True`, `class`, `import`.

Valid	Invalid	Why invalid
<code>user_name</code>	<code>user name</code>	Contains a space
<code>_secret</code>	<code>2nd_try</code>	Starts with a digit
<code>port2</code>	<code>port-number</code>	Hyphen not allowed
<code>ip_address</code>	<code>class</code>	Reserved keyword
<code>Total</code>	<code>total\$</code>	<code>\$</code> not allowed

To see all reserved keywords:

```
python
```

```
import keyword
```

```
print(keyword.kwlist)
```

Style conventions (not enforced, but professionals follow them)

- Use **snake_case**: lowercase words separated by underscores (`failed_login_count`).
- Choose **meaningful names**: `target_ip` is better than `x`.
- Constants (values that shouldn't change) are written in **UPPER_CASE**: `MAX_ATTEMPTS = 3`.
- Avoid names that shadow built-in functions, such as `print`, `str`, `list`, or `type`. For example, `type = "admin"` would break the `type()` function later in your program.

```
python
```

```
# Bad
```

```
x = "192.168.1.1"
```

```
a = 3
```

```
# Good
```

```
server_ip = "192.168.1.1"
```

```
max_attempts = 3
```

4. Using type()

type() is a built-in function that tells you the data type of a value or variable. It is very useful for debugging.

```
python
```

```
port = 80
```

```
cpu = 72.5
```

```
host = "localhost"
```

```
is_open = True
```

```
print(type(port))    # <class 'int'>
```

```
print(type(cpu))     # <class 'float'>
```

```
print(type(host))    # <class 'str'>
```

```
print(type(is_open)) # <class 'bool'>
```

Checking the type after conversion:

```
python
```

```
value = "443"
```

```
print(type(value))   # <class 'str'>
```

```
value = int(value)
```

```
print(type(value))   # <class 'int'>
```

Checking a type in a condition (you'll use this more after Day 5):

```
python
```

```
print(isinstance(80, int))    # True
```

```
print(isinstance("80", int)) # False
```

Debugging example: a common bug is treating text as a number.

```
python
```

```
port = "80"
```

```
print(type(port))    # <class 'str'>, so port + 1 would fail
```

```
print(int(port) + 1)  # 81
```

Putting It All Together: Scan Report Program

Create day3_program.py:

```
python
```

```
# Program: Day 3 variables and data types
```

```
# Purpose: Store and display scan information (lab data only)
```

```
target_name = "Lab-Server-01"    # str
target_ip = "192.168.56.101"     # str (private lab address)
open_ports = 3                   # int
scan_time = 2.45                 # float (seconds)
is_vulnerable = False            # bool
```

```
print("=" * 30)
print(" SCAN REPORT")
print("=" * 30)
print("Target:", target_name)
print("IP Address:", target_ip)
print("Open ports:", open_ports)
print("Scan time:", scan_time, "seconds")
print("Vulnerable:", is_vulnerable)
```

```
print()
print("Type of target_name:", type(target_name))
print("Type of open_ports:", type(open_ports))
print("Type of scan_time:", type(scan_time))
print("Type of is_vulnerable:", type(is_vulnerable))
```

Output:

```
=====
SCAN REPORT
=====
Target: Lab-Server-01
IP Address: 192.168.56.101
Open ports: 3
Scan time: 2.45 seconds
```

Vulnerable: False

Type of target_name: <class 'str'>

Type of open_ports: <class 'int'>

Type of scan_time: <class 'float'>

Type of is_vulnerable: <class 'bool'>

This is sample data for a lab machine you own. Nothing is actually being scanned.

Common Mistakes

Mistake	Problem	Fix
2port = 80	Starts with a digit	port2 = 80
my port = 80	Space in name	my_port = 80
is_open = true	Lowercase true	True
print(Port) when you defined port	NameError (case-sensitive)	Match the exact spelling
"Age: " + 25	TypeError	"Age: " + str(25)
Using a variable before assigning it	NameError	Assign first, then use

Practice Tasks

1. Create variables for your name (str), age (int), height in metres (float), and whether you're a student (bool). Print them all.
2. Print the type of each variable using type().
3. Create port = "8080", convert it to an integer, add 1, and print the result.
4. Find out what bool(""), bool("False"), and bool(0.0) return. Can you explain why?
5. Write down five invalid variable names and the rule each one breaks, then test them in Python to see the errors.
6. Create a small "login attempt" program with variables username, attempts_left, is_locked, and print a formatted summary.

Day 5: Operators

1. Arithmetic Operators

Arithmetic operators perform math on numbers. You saw a few on Day 3; here is the full set.

Operator	Name	Example	Result
+	Addition	10 + 3	13
-	Subtraction	10 - 3	7
*	Multiplication	10 * 3	30
/	Division (always float)	10 / 4	2.5
//	Floor division	10 // 4	2
%	Modulus (remainder)	10 % 4	2
**	Exponent (power)	2 ** 5	32

python

```
total_ports = 1024
```

```
scanned = 256
```

```
print(total_ports - scanned)  # 768
```

```
print(total_ports / scanned)  # 4.0
```

```
print(total_ports // 300)     # 3
```

```
print(total_ports % 300)      # 124
```

```
print(2 ** 8)                 # 256
```

Handy uses of %:

python

```
number = 14
```

```
print(number % 2 == 0)  # True, so the number is even
```

Modulus is also used to wrap values around, such as cycling through characters in encryption exercises.

Operator precedence (the order Python calculates):

1. () parentheses
2. **
3. *, /, //, %
4. +, -

python

```
print(2 + 3 * 4)    # 14 (multiplication first)
```

```
print((2 + 3) * 4)  # 20 (parentheses first)
```

```
print(2 ** 3 ** 2)  # 512 (** is evaluated right to left: 3**2=9, then 2**9)
```

When in doubt, use parentheses. They also make code easier to read.

Assignment shortcuts (compound operators):

Shortcut	Same as
<code>x += 5</code>	<code>x = x + 5</code>
<code>x -= 5</code>	<code>x = x - 5</code>
<code>x *= 2</code>	<code>x = x * 2</code>
<code>x /= 2</code>	<code>x = x / 2</code>
<code>x //= 2</code>	<code>x = x // 2</code>
<code>x %= 3</code>	<code>x = x % 3</code>
<code>x **= 2</code>	<code>x = x ** 2</code>

python

```
failed_logins = 0
```

```
failed_logins += 1
```

```
failed_logins += 1
```

```
print(failed_logins)  # 2
```

Division by zero crashes the program:

python

```
print(10 / 0)  # ZeroDivisionError
```

Floating-point note:

python

```
print(0.1 + 0.2)  # 0.30000000000000004
```

Computers store decimals in binary, so tiny rounding differences appear. Use `round(0.1 + 0.2, 2)` to display 0.3.

Operators on strings:

python

```
print("ab" + "cd")  # abcd
```

```
print("-" * 10)     # -----
```

2. Comparison Operators

Comparison operators compare two values and always return a **bool** (True or False).

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	7 > 10	False
<	Less than	7 < 10	True
>=	Greater than or equal	5 >= 5	True
<=	Less than or equal	4 <= 3	False

python

```
port = 443

print(port == 443)    # True

print(port != 80)     # True

print(port > 1024)    # False

print(port <= 1023)   # True (a "well-known" port range)
```

The classic mistake: = vs ==

- **= assigns** a value: port = 80
- **== compares** two values: port == 80

Comparing strings (case-sensitive):

python

```
print("admin" == "admin")    # True

print("admin" == "Admin")    # False

print("admin" == "Admin".lower()) # True

print("apple" < "banana")    # True (alphabetical order)
```

Chained comparisons (a Python favorite):

python

```
port = 8080

print(1024 <= port <= 49151) # True: inside the registered port range
```

Careful with types:

```
python
```

```
print(80 == "80")    # False: int vs str
```

```
print(80 == int("80")) # True
```

Membership and identity operators (bonus):

```
python
```

```
print("22" in "22,80,443")    # True
```

```
print("admin" not in ["root", "guest"]) # True
```

3. Logical Operators

Logical operators combine boolean results.

Operator	Meaning	Rule
and	Both must be true	True and True gives True, anything else gives False
or	At least one is true	Only False or False gives False
not	Reverses the value	not True gives False

Truth tables:

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

```
python
```

```
age = 20
```

```
has_id = True
```

```
print(age >= 18 and has_id)  # True
```

```
print(age < 18 or has_id)    # True
```

```
print(not has_id)            # False
```

Security-flavored example:

```
python
```

```
failed_attempts = 6
```

```
is_admin = False
```

```
lock_account = failed_attempts > 5 and not is_admin  
print(lock_account)  # True
```

4. Simple Expressions

An **expression** is any combination of values, variables, and operators that produces a result. You can mix all three operator types.

```
python
```

```
open_ports = 5
```

```
total_ports = 20
```

```
percent_open = open_ports / total_ports * 100
```

```
is_risky = percent_open > 20 or open_ports > 10
```

```
print(percent_open)  # 25.0
```

```
print(is_risky)      # True
```

Reading an expression step by step:

```
python
```

```
password_length = 10
```

```
has_number = True
```

```
is_strong = password_length >= 8 and has_number
```

1. password_length >= 8 becomes True
2. True and has_number becomes True and True
3. The result is True

Expressions inside f-strings:

```
python
```

```
a, b = 12, 5
```

```
print(f"{a} // {b} = {a // b}, remainder {a % b}")
```

```
print(f"Is {a} greater than {b}? {a > b}")
```

Putting It All Together: Password Policy Checker

Create day5_program.py:

python

Program: Day 5 operators

Purpose: Check a password against a simple policy (practice only)

```
print("=" * 34)
```

```
print(" PASSWORD POLICY CHECKER")
```

```
print("=" * 34)
```

```
password = input("Enter a test password: ")
```

```
length = len(password)
```

```
long_enough = length >= 8
```

```
not_too_long = length <= 64
```

```
has_digit_check = any(ch.isdigit() for ch in password)
```

```
not_common = password.lower() != "password"
```

```
meets_policy = long_enough and not_too_long and has_digit_check and not_common
```

```
print()
```

```
print(f"Length: {length}")
```

```
print(f"At least 8 characters: {long_enough}")
```

```
print(f"Contains a digit: {has_digit_check}")
```

```
print(f"Not the word 'password': {not_common}")
```

```
print(f"Meets policy: {meets_policy}")
```

Sample run:

```
=====
```

```
    PASSWORD POLICY CHECKER
```

```
=====
```

```
Enter a test password: Lab2026key
```

```
Length: 10
```

At least 8 characters: True

Contains a digit: True

Not the word 'password': True

Meets policy: True

Notes: len() returns the number of characters, and any(...) is a preview of loops. Use throwaway passwords here, never your real ones. Real policies also check against lists of breached passwords, and length matters more than complexity.

Common Mistakes

Mistake	Problem	Fix
if x = 5 style thinking	= assigns, it does not compare	Use ==
10 / 2 expecting 5	/ returns 5.0	Use // for whole numbers
5 > 3 and < 10	Incomplete second comparison	5 > 3 and 5 < 10 or 3 < 5 < 10
"5" > 3	TypeError comparing str and int	Convert first: int("5") > 3
True and false	false is not defined	Use capital False
Forgetting parentheses in mixed and/or	Unexpected results	Add ()

Practice Tasks

1. Ask for two numbers and print the result of all seven arithmetic operators in formatted sentences.
2. Ask for a number and print whether it is even (use %).
3. Ask for a port number and print whether it is in the well-known range (0 to 1023) using a chained comparison.
4. Predict the results, then test: not (5 > 3 and 2 > 4), 10 % 3 == 1, 2 + 3 * 4 ** 2.
5. Ask for age and whether the person has a membership (yes/no), then print True or False for "gets discount" if age is under 18 or over 60, and has membership.
6. Extend the password checker: add a rule that the password must not equal the username.

Want me to continue with Day 6 (conditions: if, elif, else), where these operators start making decisions, or compile Days 1 to 5 into a Word document or slide deck?

Day 6: Strings

1. Creating and Joining Strings

A **string** is a sequence of characters: letters, digits, spaces, and symbols. Strings are one of the most used types in security work, since usernames, IP addresses, URLs, log lines, and file paths are all text.

Creating strings:

```
python

username = "admin"

protocol = 'HTTPS'

log_line = "Failed login from 192.168.1.5"
```

Single and double quotes work the same. Pick one style and stay consistent. Use the other kind when the text contains a quote:

```
python

print("It's a secure server")

print('He said "access denied"')
```

Multi-line strings use triple quotes:

```
python

report = """Scan Report

Target: Lab-Server-01

Status: Safe"""

print(report)
```

Empty string: "" is a valid string with no characters.

Escape characters (recap):

```
python

print("Name:\tAlex\nRole:\tAnalyst")

print("C:\\Users\\lab")      # \\ prints one backslash

print(r"C:\Users\lab")      # r"..." (raw string) keeps backslashes as typed
```

Joining strings:

1. **Concatenation with +** (both sides must be strings):

```
python

first = "Cyber"

second = "Security"
```

```
print(first + second)      # CyberSecurity

print(first + " " + second) # Cyber Security
```

2. Repetition with *:

```
python

print("=" * 20)

print("ab" * 3)      # ababab
```

3. f-strings (the cleanest approach):

```
python

host = "lab-server"

port = 22

print(f"Connecting to {host}:{port}")
```

4. join(), which glues a list of strings using a separator:

```
python

parts = ["192", "168", "1", "10"]

print(".".join(parts))      # 192.168.1.10

print(", ".join(["ssh", "http", "https"])) # ssh, http, https
```

Strings are immutable: you cannot change a character in place. You create a new string instead.

```
python

word = "hello"

# word[0] = "H"      # TypeError

word = "H" + word[1:] # works: Hello
```

2. String Indexing and Slicing

Indexing

Every character has a position number (**index**), starting at **0**. Negative indexes count from the end.

```
P  y  t  h  o  n
0  1  2  3  4  5    (positive index)

-6 -5 -4 -3 -2 -1   (negative index)

python

word = "Python"

print(word[0])      # P

print(word[3])      # h
```

```
print(word[-1]) # n (last character)
```

```
print(word[-2]) # o
```

Using an index that doesn't exist causes an error:

```
python
```

```
print(word[10]) # IndexError: string index out of range
```

Slicing

Slicing extracts part of a string using [start:stop]. The **start is included, the stop is excluded**.

```
python
```

```
text = "cybersecurity"
```

```
print(text[0:5]) # cyber
```

```
print(text[5:]) # security (from index 5 to the end)
```

```
print(text[:5]) # cyber (from the start up to index 5)
```

```
print(text[:]) # cybersecurity (a full copy)
```

```
print(text[-8:]) # security (last 8 characters)
```

```
print(text[:-8]) # cyber (everything except the last 8)
```

Step value [start:stop:step]:

```
python
```

```
digits = "0123456789"
```

```
print(digits[::2]) # 02468 (every 2nd character)
```

```
print(digits[1::2]) # 13579
```

```
print(digits[::-1]) # 9876543210 (reversed)
```

Slicing never raises an error for out-of-range values; it just returns what exists:

```
python
```

```
print("abc"[0:100]) # abc
```

Practical examples

```
python
```

```
ip = "192.168.1.10"
```

```
print(ip[:3]) # 192 (first octet, because it has 3 digits)
```

```
filename = "report.pdf"
```

```
print(filename[-3:]) # pdf (file extension)
```

```
email = "alex@example.com"
```

```
print(email[:4])    # alex
```

Useful related tools:

```
python
```

```
text = "Failed login from 10.0.0.5"
```

```
print("login" in text)    # True
```

```
print(text.find("from"))  # 13 (index where "from" starts, or -1 if not found)
```

```
print(text.count("l"))    # 3
```

3. lower(), upper(), and strip()

Strings have built-in **methods**: functions attached to the string, called with a dot. Each method **returns a new string** and leaves the original unchanged.

lower() and upper()

```
python
```

```
name = "Alex Smith"
```

```
print(name.lower())    # alex smith
```

```
print(name.upper())    # ALEX SMITH
```

```
print(name)            # Alex Smith (unchanged)
```

Why it matters: comparisons are case-sensitive, so normalize first.

```
python
```

```
answer = "YES"
```

```
print(answer == "yes")    # False
```

```
print(answer.lower() == "yes")    # True
```

Related: `title()` capitalizes each word ("alex smith".`title()` gives Alex Smith) and `capitalize()` capitalizes only the first letter.

strip()

`strip()` removes whitespace (spaces, tabs, newlines) from **both ends**. It does not touch spaces in the middle.

```
python
```

```
raw = "  admin  "
```

```
print(f"[{raw.strip()}]")    # [admin]
```

```
print(f"[{raw.lstrip()}]")    # [admin ] (left only)
```

```
print(f"[{raw.rstrip()}]") # [ admin] (right only)
```

You can also strip specific characters:

```
python
```

```
print("***alert***".strip("*")) # alert
```

Combining methods (method chaining)

```
python
```

```
user_input = " AdMiN "
```

```
clean = user_input.strip().lower()
```

```
print(clean) # admin
```

This pattern is the standard way to clean input:

```
python
```

```
username = input("Username: ").strip().lower()
```

Other handy methods (preview)

```
python
```

```
text = "port 22 is open"
```

```
print(text.replace("22", "443")) # port 443 is open
```

```
print(text.split()) # ['port', '22', 'is', 'open']
```

```
print(text.startswith("port")) # True
```

```
print("12345".isdigit()) # True
```

4. String Length Using len()

len() returns the number of characters in a string, **including spaces and symbols**.

```
python
```

```
print(len("Python")) # 6
```

```
print(len("hello world")) # 11 (the space counts)
```

```
print(len("")) # 0
```

```
print(len(" hi ")) # 6
```

```
print(len(" hi ".strip())) # 2
```

The last character is always at index len(text) - 1:

```
python
```

```
word = "security"
```

```
print(word[len(word) - 1]) # y
```

```
print(word[-1])          # y (simpler)
```

Practical uses:

```
python
```

```
password = "Lab2026key"
```

```
print(len(password) >= 8)    # True: meets minimum length
```

```
pin = "4821"
```

```
print(len(pin) == 4 and pin.isdigit())  # True: valid 4-digit PIN
```

len() also works on lists, tuples, and dictionaries, which you will use later.

Putting It All Together: Username Cleaner and Validator

Create day6_program.py:

```
python
```

```
# Program: Day 6 strings
```

```
# Purpose: Clean and check a username (practice only)
```

```
print("=" * 34)
```

```
print(" USERNAME CLEANER")
```

```
print("=" * 34)
```

```
raw_name = input("Enter a username: ")
```

```
clean_name = raw_name.strip().lower()
```

```
length = len(clean_name)
```

```
valid_length = 4 <= length <= 12
```

```
no_spaces = " " not in clean_name
```

```
print()
```

```
print(f"Raw input:  [{raw_name}]")
```

```
print(f"Cleaned:    [{clean_name}]")
```

```
print(f"Length:     {length}")
```

```
print(f"First letter: {clean_name[0:1].upper()}")
```

```
print(f"Last 2 chars: {clean_name[-2:]}")
print(f"Reversed: {clean_name[::-1]}")
print(f"Valid length (4-12): {valid_length}")
print(f"No spaces inside: {no_spaces}")
```

Sample run:

```
=====
USERNAME CLEANER
=====
```

Enter a username: AlexSmith

Raw input: [AlexSmith]

Cleaned: [alexsmith]

First letter: A

Last 2 chars: th

Reversed: htimsxela

Length: 9

Valid length (4-12): True

No spaces inside: True

Note: `clean_name[0:1]` is used instead of `clean_name[0]` so an empty input doesn't crash the program.

(In the sample, lines appear in the order the code prints them; Length comes before First letter in the actual output.)

Practice Tasks

1. Store "Cybersecurity" in a variable and print its first letter, last letter, and length.
2. Using slicing, print "Cyber" and "security" separately from "Cybersecurity".
3. Print "Python" reversed using a slice.
4. Ask for a name with extra spaces and mixed case. Print it cleaned with `strip()` and `title()`.
5. Given `ip = "192.168.1.10"`, print the last octet using slicing, then try `ip.split(".")` and compare.
6. Ask for a filename (for example `report.pdf`) and print its extension using slicing. Then check whether it ends with `.pdf` using `endswith()`.
7. Write a program that asks for a password and reports whether its length is at least 8, without printing the password itself.

8. Take " FAILED LOGIN from 10.0.0.5 ", clean it, convert it to lowercase, and print how many characters it has.

Want me to continue with Day 7 (conditions: if, elif, else), or compile Days 1 to 6 into a Word document or slide deck?

Day 7: Conditional Statements

1. if, elif, and else

So far your programs ran every line from top to bottom. **Conditional statements** let a program make decisions: run some code only when a condition is True.

The if statement

```
python
```

```
port = 22
```

```
if port == 22:
```

```
    print("This is the SSH port")
```

Anatomy:

if condition:

code that runs only if the condition is True

- The condition is any expression that gives True or False.
- The line ends with a **colon :**.
- The code to run is **indented** underneath.

If the condition is False, Python skips the indented block.

if + else

else runs when the condition is False.

```
python
```

```
attempts = 6
```

```
if attempts > 5:
```

```
    print("Account locked")
```

```
else:
```

```
    print("Try again")
```

if + elif + else

elif ("else if") lets you test several conditions in order. Python checks from top to bottom and runs **only the first block that is true**.

```
python
```

```
score = 72
```

```
if score >= 90:
```

```
    grade = "A"
```

```
elif score >= 75:
```

```
    grade = "B"
```

```
elif score >= 60:
```

```
    grade = "C"
```

```
else:
```

```
    grade = "F"
```

```
print(f"Grade: {grade}") # Grade: C
```

Order matters. If you tested `score >= 60` first, a score of 95 would get "C", because the first true condition wins.

elif chain vs. separate if statements

```
python
```

```
number = 15
```

```
# Separate ifs: each one is checked independently
```

```
if number > 10:
```

```
    print("Greater than 10")
```

```
if number > 5:
```

```
    print("Greater than 5")
```

```
# Prints both lines
```

```
# elif chain: stops at the first match
```

```
if number > 10:
```

```
    print("Greater than 10")
```

```
elif number > 5:
```

```
    print("Greater than 5")
```

```
# Prints only the first line
```

Use elif when the choices are mutually exclusive, and separate ifs when several things can be true at once.

Combining conditions

Use the logical operators from Day 5:

```
python
```

```
failed_attempts = 6
```

```
is_admin = False
```

```
if failed_attempts > 5 and not is_admin:
```

```
    print("Lock the account")
```

```
role = "guest"
```

```
if role == "admin" or role == "root":
```

```
    print("Privileged user")
```

Nested conditions

An if can sit inside another if:

```
python
```

```
logged_in = True
```

```
role = "admin"
```

```
if logged_in:
```

```
    if role == "admin":
```

```
        print("Welcome to the admin panel")
```

```
    else:
```

```
        print("Welcome, user")
```

```
else:
```

```
    print("Please log in first")
```

Keep nesting shallow. Often and makes the code flatter and easier to read.

Truthiness shortcut

Python treats some values as False: 0, 0.0, "" (empty string), and empty collections. Everything else is True.

python

```
username = ""
```

```
if username:
```

```
    print("Name entered")
```

```
else:
```

```
    print("Name is empty")    # this runs
```

One-line conditional (bonus)

python

```
status = "open" if port == 80 else "closed"
```

Handy for simple cases. Use a normal if block when the logic gets longer.

2. Indentation in Python

Most languages use braces { } to group code. Python uses **indentation** (the spaces at the start of a line). It is part of the syntax, not just style.

python

```
if True:
```

```
    print("Inside the block")
```

```
    print("Still inside")
```

```
print("Outside the block")
```

Rules:

- Use **4 spaces** per level (the Python standard).
- All lines in the same block must have the **same** indentation.
- Don't mix tabs and spaces. In VS Code, pressing Tab inserts spaces for Python files by default.
- A block must contain at least one statement. Use `pass` as a placeholder:

python

```
if port == 80:
```

```
    pass    # TODO: add code later
```

What a block's end looks like: the block ends when the indentation goes back to the previous level.

```
python
```

```
age = 20
```

```
if age >= 18:
```

```
    print("Adult")
```

```
    print("Can create an account")
```

```
print("This line always runs") # not indented, so not part of the if
```

Common indentation errors:

```
python
```

```
# 1. Missing indent
```

```
if port == 80:
```

```
    print("Web")
```

```
# IndentationError: expected an indented block
```

```
# 2. Inconsistent indent
```

```
if port == 80:
```

```
    print("Web")
```

```
        print("Port 80")
```

```
# IndentationError: unexpected indent
```

```
# 3. Missing colon
```

```
if port == 80
```

```
    print("Web")
```

```
# SyntaxError: expected ':'
```

Indentation changes meaning. These two programs behave differently:

```
python
```

```
# Version A
```

```
attempts = 3
```

```
if attempts > 5:
```

```
    print("Locked")
```

```
print("Done")    # always prints
```

```
# Version B
```

```
attempts = 3
```

```
if attempts > 5:
```

```
    print("Locked")
```

```
    print("Done")    # prints only if attempts > 5
```

3. Comparison Conditions

Every condition is built from the comparison operators you learned on Day 5.

Operator	Meaning	Example
==	Equal	status == "open"
!=	Not equal	role != "guest"
>	Greater than	attempts > 3
<	Less than	length < 8
>=	Greater than or equal	age >= 18
<=	Less than or equal	port <= 1023

Examples:

```
python
```

```
port = 443
```

```
if port == 80:
```

```
    print("HTTP")
```

```
elif port == 443:
```

```
    print("HTTPS")
```

```
else:
```

```
    print("Unknown service")
```

Chained comparisons:

```
python
```

```
if 1 <= port <= 1023:
```

```
    print("Well-known port")
```

Membership with in:

python

```
if port in (80, 443, 8080):  
    print("Web-related port")
```

```
if "admin" in username:  
    print("Name contains 'admin'")
```

String comparisons are case-sensitive, so normalize first:

python

```
choice = input("Continue? (yes/no): ").strip().lower()
```

```
if choice == "yes":  
    print("Continuing...")  
elif choice == "no":  
    print("Stopping.")  
else:  
    print("Please type yes or no.")
```

Watch your types:

python

```
port_text = input("Port: ")    # a string  
  
if port_text == 80:            # always False: str vs int  
    print("Match")
```

```
if int(port_text) == 80:        # correct (after converting)  
    print("Match")
```

= vs ==: if port = 80: is a `SyntaxError`. Use `==` to compare.

4. Input Validation

Input validation means checking that data is acceptable before using it. Users make typos, and attackers deliberately send unexpected input. In security work, the rule is: **never trust input**. Many real-world attacks, such as SQL injection, exist because programs used unchecked input.

Common checks

1. Is it empty?

python

```
name = input("Name: ").strip()
```

```
if name == "":
```

```
    print("Name cannot be empty")
```

2. Is it the right kind of data?

```
python
```

```
text = input("Enter port: ").strip()
```

```
if text.isdigit():
```

```
    port = int(text)
```

```
else:
```

```
    print("Digits only, please")
```

This prevents the ValueError crash you saw on Day 4. Note that `isdigit()` returns False for negative numbers ("-5") and decimals ("3.5"). Proper error handling with `try/except` comes later.

3. Is it within a valid range?

```
python
```

```
if 0 <= port <= 65535:
```

```
    print("Valid port")
```

```
else:
```

```
    print("Port must be between 0 and 65535")
```

4. Is it one of the allowed values?

```
python
```

```
protocol = input("Protocol (tcp/udp: ").strip().lower()
```

```
if protocol in ("tcp", "udp"):
```

```
    print(f"Using {protocol.upper()}")
```

```
else:
```

```
    print("Invalid protocol")
```

5. Is the length acceptable?

```
python
```

```
password = input("New password: ")
```

```
if len(password) < 8:
```

```
    print("Too short")
```

Validate in the right order

Check the cheap, safe things first. Never convert before checking:

```
python
```

```
text = input("Enter port: ").strip()
```

```
if text == "":
```

```
    print("Error: empty input")
```

```
elif not text.isdigit():
```

```
    print("Error: digits only")
```

```
elif int(text) > 65535:
```

```
    print("Error: port too large")
```

```
else:
```

```
    print(f"Port {int(text)} accepted")
```

The `int()` call only runs after `isdigit()` has confirmed it's safe. Because `elif` stops at the first match, this order protects the program.

Putting It All Together: Port Checker with Validation

Create `day7_program.py`:

```
python
```

```
# Program: Day 7 conditional statements
```

```
# Purpose: Validate a port number and describe it (practice only)
```

```
print("=" * 34)
```

```
print(" PORT INFO CHECKER")
```

```
print("=" * 34)
```

```
text = input("Enter a port number (0-65535): ").strip()
```

```
# Step 1: validate
```

```
if text == "":
```

```
    print("Error: input cannot be empty.")
```

```
elif not text.isdigit():

    print("Error: please enter digits only.")

elif int(text) > 65535:

    print("Error: port must be between 0 and 65535.")

else:

    # Step 2: input is valid, so convert and classify

    port = int(text)

    if port <= 1023:

        category = "Well-known"

    elif port <= 49151:

        category = "Registered"

    else:

        category = "Dynamic/private"

    print()

    print(f"Port: {port}")

    print(f"Range: {category}")

    # Step 3: common services

    if port == 22:

        print("Commonly used for: SSH (remote login)")

    elif port == 80:

        print("Commonly used for: HTTP (web)")

    elif port == 443:

        print("Commonly used for: HTTPS (secure web)")

    elif port == 21:

        print("Commonly used for: FTP (file transfer)")

    else:

        print("No common service in our short list")
```

Sample runs:

Enter a port number (0-65535): 443

Port: 443

Range: Well-known

Commonly used for: HTTPS (secure web)

Enter a port number (0-65535): 99999

Error: port must be between 0 and 65535.

Enter a port number (0-65535): abc

Error: please enter digits only.

Bonus mini-program: a login simulator (practice only):

```
python
```

```
# Hard-coded credentials are fine for a lesson, never for real systems
```

```
correct_user = "student"
```

```
correct_pass = "Lab2026!"
```

```
user = input("Username: ").strip()
```

```
pwd = input("Password: ")
```

```
if user == "" or pwd == "":
```

```
    print("Both fields are required.")
```

```
elif user == correct_user and pwd == correct_pass:
```

```
    print("Access granted")
```

```
else:
```

```
    print("Access denied")    # don't reveal which field was wrong
```

Real systems never store plain-text passwords. They store hashes, which you will meet later in the course. Also note that the "denied" message doesn't say whether the username or the password was wrong. Specific messages help attackers.

Common Mistakes

Mistake	Problem	Fix
if port = 80:	= assigns, SyntaxError	if port == 80:
if port == 80 (no colon)	SyntaxError	Add :

Forgetting to indent the block	IndentationError	Indent 4 spaces
Mixing tabs and spaces	TabError or odd behavior	Use 4 spaces only
if x == 1 or 2:	Always True (2 is truthy)	if x == 1 or x == 2: or if x in (1, 2):
elif without a preceding if	SyntaxError	Start with if
Comparing input() text to a number	Always False	Convert with int() first
Wrong order in an elif chain	Wrong branch runs	Put most specific conditions first
Calling int() before validating	ValueError crash	Check with isdigit() first
Case mismatch ("YES" vs "yes")	Valid input rejected	Use .strip().lower()

Practice Tasks

1. Ask for a number and print whether it is positive, negative, or zero.
2. Ask for an age and print "Child" (under 13), "Teen" (13 to 17), "Adult" (18 to 64), or "Senior" (65+).
3. Ask for a password and print "Weak" if shorter than 8 characters, "Medium" if 8 to 11, and "Strong" if 12 or more.
4. Ask for a yes/no answer. Accept Y, yes, YES, and so on, and print an error for anything else.
5. Ask for a port number and validate it fully (non-empty, digits only, between 0 and 65535). Then print "Privileged port" if it is below 1024.
6. Write a simple IP octet checker: ask for a number and say whether it is a valid IPv4 octet (0 to 255).
7. Extend the login simulator so it also rejects usernames shorter than 3 characters with a specific message *before* checking the credentials.
8. Predict the output, then test:

python

```

x = 7

if x > 5:
    print("A")

elif x > 3:
    print("B")

if x > 6:
    print("C")

else:
    print("D")

```

9. Deliberately break a program: remove a colon, then change an indent. Read each error message and note the line number.

Want me to continue with Day 8 (loops: for and while), or compile Days 1 to 7 into a Word document or slide deck?

Day 8: Loops

- for loop
- while loop
- range() function
- break and continue

Day 8: Loops

1. The for Loop

A **loop** repeats code without you writing it many times. Security tasks are full of repetition: checking many ports, reading every line of a log, testing a list of hosts you are authorized to scan.

A **for loop** goes through a sequence one item at a time.

```
python
```

```
for port in [22, 80, 443]:
```

```
    print(f"Checking port {port}")
```

Output:

```
Checking port 22
```

```
Checking port 80
```

```
Checking port 443
```

Anatomy:

for variable in sequence:

code that runs once per item

- variable takes the next item on each pass (called an **iteration**).
- The line ends with a colon, and the body is indented (Day 7 rules).

Looping over a string (one character at a time):

```
python
```

```
for ch in "SSH":
```

```
    print(ch)    # S, then S, then H
```

Looping over a list of strings:

python

```
services = ["ssh", "http", "https"]
```

```
for name in services:
```

```
    print(name.upper())
```

Using the loop to build a result:

python

```
total = 0
```

```
for number in [10, 20, 30]:
```

```
    total += number
```

```
print(total)      # 60
```

Counting matches:

python

```
log_lines = ["login ok", "login failed", "login failed", "logout"]
```

```
failed = 0
```

```
for line in log_lines:
```

```
    if "failed" in line:
```

```
        failed += 1
```

```
print(f"Failed logins: {failed}") # 2
```

Getting the position too, with enumerate():

python

```
for index, name in enumerate(services, start=1):
```

```
    print(f"{index}. {name}")
```

Output:

1. ssh

2. http

3. https

Lists are written with square brackets and are covered fully on Day 9. For now, treat them as a way to hold several values.

2. The while Loop

A **while loop** repeats **as long as a condition is True**. Use it when you don't know in advance how many repetitions you need.

```
python
```

```
count = 1
```

```
while count <= 3:
```

```
    print(f"Attempt {count}")
```

```
    count += 1
```

Output:

Attempt 1

Attempt 2

Attempt 3

Anatomy:

```
while condition:
```

```
    code that repeats
```

```
    (something here must eventually make the condition False)
```

Three parts of a safe while loop:

1. **Initialize** a variable before the loop (count = 1).
2. **Test** it in the condition (count <= 3).
3. **Update** it inside the loop (count += 1).

Infinite loops: if the condition never becomes False, the program runs forever. This happens when you forget the update step:

```
python
```

```
count = 1
```

```
while count <= 3:
```

```
    print("Stuck")    # count never changes!
```

Stop a runaway program with Ctrl + C in the terminal.

Typical use: repeat until the input is valid

```
python
```

```
text = input("Enter a port (digits only): ").strip()
```

```
while not text.isdigit():
```

```
print("Invalid input.")

text = input("Enter a port (digits only): ").strip()
```

```
print(f"Port accepted: {int(text)}")
```

Limited login attempts (practice only):

```
python
```

```
correct_pin = "4821"
```

```
attempts_left = 3
```

```
while attempts_left > 0:
```

```
    pin = input("Enter PIN: ")
```

```
    if pin == correct_pin:
```

```
        print("Access granted")
```

```
        break
```

```
    attempts_left -= 1
```

```
    print(f"Wrong PIN. Attempts left: {attempts_left}")
```

```
else:
```

```
    print("Account locked")
```

Two new ideas appear here: `break` (Section 4) and the `loop else`. A loop's `else` block runs only if the loop finished **without** hitting `break`. It is a bonus feature, so don't worry if it feels unusual at first.

for vs. while:

Use	When
for	You know what to loop over (a list, a range, a string)
while	You loop until something changes (valid input, a counter reaching a limit)

3. The `range()` Function

`range()` generates a sequence of numbers, which makes it perfect for `for` loops that need to run a set number of times.

Three forms:

Form	Meaning	Example	Numbers produced
<code>range(stop)</code>	0 up to stop - 1	<code>range(5)</code>	0, 1, 2, 3, 4
<code>range(start, stop)</code>	start up to stop - 1	<code>range(2, 6)</code>	2, 3, 4, 5

range(start, stop, step)	jump by step	range(0, 10, 2)	0, 2, 4, 6, 8
--------------------------	--------------	-----------------	---------------

The stop value is never included, just like slicing.

python

```
for i in range(5):
```

```
    print(i)          # 0 1 2 3 4
```

```
for i in range(1, 6):
```

```
    print(i)          # 1 2 3 4 5
```

```
for i in range(10, 0, -2):
```

```
    print(i)          # 10 8 6 4 2 (counting down)
```

Repeat something N times:

python

```
for _ in range(3):
```

```
    print("Scanning...")
```

The name `_` is a convention meaning "I don't need this variable."

Scanning a range of ports in a lab (this only prints; nothing connects to anything):

python

```
for port in range(20, 26):
```

```
    print(f"Would check port {port}")
```

Generating a range of lab IP addresses (private lab network):

python

```
for last_octet in range(1, 6):
```

```
    print(f"192.168.56.{last_octet}")
```

Output: 192.168.56.1 through 192.168.56.5.

Using range with len() to loop by position:

python

```
word = "Python"
```

```
for i in range(len(word)):
```

```
    print(i, word[i])
```

Multiplication table:

python

```
for i in range(1, 6):
```

```
    print(f"7 x {i} = {7 * i}")
```

Tips:

- To convert to a list and look at it: `print(list(range(5)))` gives `[0, 1, 2, 3, 4]`.
- `range()` only works with **integers**. `range(0, 1, 0.1)` raises a `TypeError`.
- `range(5, 1)` produces nothing, because the default step is `+1` and it can't count up from 5 to 1.

4. **break and continue**

These two keywords change how a loop runs.

break: exit the loop immediately

python

```
for port in [21, 22, 80, 443]:
```

```
    if port == 80:
```

```
        print("Found port 80, stopping")
```

```
        break
```

```
    print(f"Checking {port}")
```

Output:

Checking 21

Checking 22

Found port 80, stopping

Common use: stop searching once you've found what you want.

python

```
users = ["alice", "bob", "carol"]
```

```
found = False
```

```
for u in users:
```

```
    if u == "bob":
```

```
        found = True
```

```
        break
```

```
print(f"bob exists: {found}")
```

continue: skip to the next iteration

continue skips the rest of the current pass and jumps to the next item.

python

```
for number in range(1, 8):
```

```
    if number % 2 == 0:
```

```
        continue          # skip even numbers
```

```
    print(number)          # 1 3 5 7
```

Common use: ignore items you don't care about.

python

```
log_lines = ["login ok", "", "login failed", "", "logout"]
```

```
for line in log_lines:
```

```
    if line == "":
```

```
        continue          # skip blank lines
```

```
    print(f"Processing: {line}")
```

break vs. continue

Keyword	Effect
break	Ends the whole loop
continue	Ends only the current pass , then the loop carries on

Infinite loop with a controlled exit

while True: loops forever, and break is the way out. This is a popular menu pattern:

python

```
while True:
```

```
    command = input("Command (scan/help/quit): ").strip().lower()
```

```
    if command == "quit":
```

```
        print("Goodbye")
```

```
        break
```

```
    elif command == "help":
```

```
        print("Available: scan, help, quit")
```

```
    elif command == "scan":
```

```
    print("Lab demo: no real scan runs")

else:

    print("Unknown command")
```

Nested loops (bonus)

A loop inside a loop. The inner loop runs fully for each pass of the outer loop. break only exits the loop it is inside.

python

```
for host in range(1, 3):

    for port in [22, 80]:

        print(f"Host {host} port {port}")
```

Output:

Host .1 port 22

Host .1 port 80

Host .2 port 22

Host .2 port 80

Putting It All Together: Failed Login Detector

This program analyzes **sample log data stored in the script**. It doesn't touch any real system.

Create day8_program.py:

python

Program: Day 8 loops

Purpose: Analyze sample log lines for repeated failed logins (practice only)

```
log_lines = [

    "10:01 login ok user=alice",

    "10:02 login failed user=bob",

    "",

    "10:03 login failed user=bob",

    "10:04 login ok user=carol",

    "10:05 login failed user=bob",

    "10:06 login failed user=bob",

]
```

```

LIMIT = 3

failed_count = 0

processed = 0

print("=" * 34)

print(" LOG ANALYSIS")

print("=" * 34)

for line in log_lines:

    if line == "":

        continue          # skip blank lines

    processed += 1

    if "failed" in line:

        failed_count += 1

        print(f"Failure #{failed_count}: {line}")

    if failed_count >= LIMIT:

        print(f"ALERT: {LIMIT} failed logins reached, stopping analysis")

        break

print()

print(f"Lines processed: {processed}")

print(f"Failed logins found: {failed_count}")

```

Output:

```

=====

LOG ANALYSIS

=====

Failure #1: 10:02 login failed user=bob

Failure #2: 10:03 login failed user=bob

```

Failure #3: 10:05 login failed user=bob

ALERT: 3 failed logins reached, stopping analysis

Lines processed: 5

Failed logins found: 3

This is the same idea real defenders use to detect brute-force attempts: count failures and raise an alert at a threshold.

Practice Tasks

1. Print the numbers 1 to 10 using a for loop and range().
2. Print all even numbers from 2 to 20.
3. Count down from 10 to 1, then print "Liftoff!"
4. Ask for a number and print its multiplication table (1 to 10) using f-strings.
5. Calculate the sum of numbers from 1 to 100 using a loop. Check it against the formula $100 * 101 // 2$.
6. Given ports = [21, 22, 23, 80, 443, 3389], print only the ports below 1024, using continue to skip the others.
7. Given the same list, stop the loop at the first port that equals 80 and print a message.
8. Use a while loop to keep asking for a password until it is at least 8 characters long.
9. Build the menu program from Section 4 and add a status command that prints how many commands have been entered so far.
10. Given text = "Failed login from 10.0.0.5", count how many vowels it contains using a for loop.
11. Using nested loops, print a small grid of host (1 to 3) and port (22, 80) combinations.
12. Predict the output, then test:

python

```
for i in range(1, 8):  
  
    if i == 3:  
  
        continue  
  
    if i == 6:  
  
        break  
  
    print(i)
```

